

整合練習 8：新的 JDBC 風格連線語法 (ADBC)

Lecture

if06 教完 `EXEC SQL...ENDEXEC` (Native SQL 的傳統寫法) 之後，官方文件就明講「新程式不要再用這個，新開發只往 ADBC 走」。這題就是教 **ADBC (ABAP Database Connectivity)**——物件導向、風格上很接近 Java JDBC (`Connection` / `Statement` / `ResultSet` 三件套) 的資料庫存取 API，`CL_SQL_CONNECTION` / `CL_SQL_STATEMENT` / `CL_SQL_RESULT_SET` 三個類別剛好一一對應。

三個核心類別，介面都已用 `sap_get_source` 直接讀出真實定義 (不是查文件轉述，是讀這套系統實際的 Kernel 類別原始碼)：

① `CL_SQL_CONNECTION`——對應 JDBC 的 `Connection`，代表一條資料庫連線：

```
class-methods get_connection
importing
    !con_name          type dbcon_name default space " 空白 = 目前登入的預設連線
    value(sharable) type flag default space
returning
    value(con_ref) type ref to cl_sql_connection
raising
    cx_sql_exception .

methods create_statement
importing
    !tab_name_for_trace type tabname optional
returning
    value(stmt_ref) type ref to cl_sql_statement .
```

`get_connection( )` 不帶參數 (或帶 'DEFAULT') 就是連目前系統的預設資料庫連線；帶一個 `dbcon_name`，就是連到 if07 教過的 **DBCO** 設定的 **Secondary Database Connection**——這正是這門課三題 (if06 Native SQL / if07 DBCO / if08 ADBC) 串起來的地方：if07 設定「連去哪裡」，if08 才是「怎麼連過去、怎麼下查詢」。有意思的驗證發現：`get_connection` 的原始碼裡，查證連線名稱是否存在的邏輯就是一句最普通的 Open SQL——

```
select single dbms into l_dbms
from dbcon
where con_name = con_name.
```

代表 **DBCON** 這張表本身完全是可以被 **ABAP** 程式正常 **SELECT** 的 (連 SAP 自己的標準類別都這樣讀)；if07 提到用資料預覽工具查 **DBCON** 被系統擋下 (`You cannot display DBCON with the standard tools`)，那個限制是資料預覽這個便利工具自己刻意加的保護，不是 Open SQL 對這張表有什麼特殊限制——這是一個重要的釐清，避免誤以為 **DBCON** 是「連 Open SQL 都讀不到」的表。

② `CL_SQL_STATEMENT`——對應 JDBC 的 `Statement`，代表一句準備要執行的 SQL：

```

methods execute_query
  importing
    !statement          type string optional
  returning
    value(result_set)    type ref to cl_sql_result_set
  raising
    cx_sql_exception
    cx_parameter_invalid .

methods execute_update
  importing
    !statement          type string optional
  returning
    value(rows_processed) type i
  raising
    cx_sql_exception
    cx_parameter_invalid .

methods execute_ddl
  importing
    !statement type string
  raising
    cx_sql_exception .

```

三支方法分工很清楚：`execute_query` (`SELECT` · 回傳一個 `CL_SQL_RESULT_SET`)、`execute_update` (`INSERT/UPDATE/DELETE` · 回傳異動筆數)、`execute_ddl` (`CREATE TABLE` 這類語法)——比 `EXEC SQL...ENDEXEC` 那種「什麼語法都塞在同一個區塊裡」清楚很多，光看呼叫的是哪個方法就知道這句 SQL 的意圖。

③ `CL_SQL_RESULT_SET`——對應 `JDBC` 的 `ResultSet`，代表查詢結果，要逐筆 `next()` 往下走：

```

methods NEXT
  returning
    value(ROWS_RET) type I
  raising
    CX_SQL_EXCEPTION
    CX_PARAMETER_INVALID_TYPE .

methods SET_PARAM_STRUCT
  importing
    !STRUCT_REF type ref to DATA
  ...

```

用法是先 `set_param_struct( )` 綁定一個 ABAP 結構當「每次 `next()` 讀到的那一列要放進哪裡」，之後 `WHILE lo_result->next( ) > 0`. 迴圈，每呼叫一次 `next()` 就把下一列資料填進綁定好的結構——這個

「先綁結構、再逐列 fetch」的模式，跟 if06 Native SQL 游標的 **OPEN/FETCH/CLOSE** 概念上是同一件事，只是包成物件導向介面。

完整寫法範例（節錄自答案程式，讀取 **SCARR**）：

```
DATA(lo_connection) = cl_sql_connection=>get_connection( ).
DATA(lo_statement)  = lo_connection->create_statement( ).

TRY.
    DATA(lo_result) = lo_statement->execute_query(
        |SELECT carrid, carrname FROM scarr WHERE mandt = '{ sy-mandt }' ORDER BY
        carrid| ).

    DATA ls_carrier TYPE ty_carrier.
    lo_result->set_param_struct( REF #( ls_carrier ) ).

    WHILE lo_result->next( ) > 0.
        WRITE: / ls_carrier-carrid, ls_carrier-carrname.
    ENDWHILE.

    lo_result->close( ).

CATCH cx_sql_exception INTO DATA(lx_sql).
    WRITE: / lx_sql->get_text( ).
ENDTRY.
```

⚠ 這裡的 **SQL** 字串是用 **ABAP** 字串模板組出來的（`|...{ sy-mandt }...|`），代表這是動態 **SQL**——跟 Open SQL 大多數情境是「靜態、編譯期就決定好」的 **SELECT** 不一樣，這帶來 SQL Injection 風險：如果字串裡任何一段是直接使用者輸入（沒有做過濾 / 參數化），就有被注入惡意 **SQL** 的風險。這題的範例用 **sy-mandt**（系統欄位，不是使用者輸入）相對安全，但真的要接使用者輸入時，應該用 **execute_query** 搭配參數化查詢（`?` 佔位符 + **set_param**），而不是把使用者輸入直接串進 **SQL** 字串——這點呼應 CLAUDE.md 對所有程式碼的通用要求：「不要引入 SQL injection」，ADBC 因為要動態組字串，是這整個課程裡最容易不小心犯這個錯的地方。

錯誤處理：**cx_sql_exception**（連線層）、**cx_parameter_invalid**（參數層）都是 Class-based Exception，用 **TRY...CATCH** 攔截，**get_text()** 拿可讀的錯誤訊息——比 Native SQL 只能看 **sy-subrc** 猜錯誤原因更明確。

學習目標

- 能寫出 ADBC 三件套（**CL_SQL_CONNECTION** / **CL_SQL_STATEMENT** / **CL_SQL_RESULT_SET**）的基本讀取流程
- 理解 **execute_query** / **execute_update** / **execute_ddl** 三個方法的分工
- 知道 ADBC 的 **SQL** 是動態字串組出來的，意識到 SQL Injection 風險，知道遇到使用者輸入該怎麼處理（參數化，不要直接字串拼接）
- 能用 **TRY...CATCH cx_sql_exception** 攔截並讀出可讀錯誤訊息
- 釐清 **DBCON** 表本身可以被 Open SQL 正常讀取，if07 遇到的擋讀限制只在特定工具層級

事前準備

不需要既有物件，這題新建一支唯讀示範程式 `ZR_IF08_ADBC_DEMO`，包含成功與失敗兩種情境。

題目需求

1. 對照答案程式的「情境①」，指出 `get_connection / create_statement / execute_query / set_param_struct / next / close` 這六個呼叫，分別對應 Lecture 講的哪個類別的哪個職責。
2. 對照「情境②」，說明為什麼故意查詢一張不存在的表可以用來測試錯誤處理邏輯——這是不是一個好的測試策略？有沒有更貼近真實情境的失敗案例值得補測？
3. 如果 Lecture 提到的 SQL 字串裡，`sy-mandt` 換成一個從畫面輸入拿到的變數 `iv_user_input`，寫法不變會有什麼風險？改寫成安全的參數化寫法（提示：`execute_query` 的 SQL 字串裡用 `?` 佔位符，呼叫前用 `set_param` 綁值）。

答案

見 `zr_if08_adbc_demo.prog.abap`（SAP 端物件 `ZR_IF08_ADBC_DEMO`，套件 `$TMP`）。已在系統上實際啟用並用 `programrun` API 執行驗證：情境①讀出 18 個航空公司代碼（跟 AMDP 課程 `am01` 用 `SQLScript` 讀到、加上正確 Client 過濾後的筆數完全一致，交叉印證這套系統 `SCARR` 的真實資料量）；情境②故意查詢不存在的表，確實觸發 `CX_SQL_EXCEPTION`，`get_text()` 讀到「The database reports an unknown database object」這則真實的資料庫錯誤訊息。

團隊實務備註

- `CL_SQL_CONNECTION / CL_SQL_STATEMENT / CL_SQL_RESULT_SET` 三個類別的介面定義都是用 `sap_get_source` 直接讀這套系統的 Kernel 類別原始碼得到的，不是查文件轉述——連「`get_connection` 內部驗證 `DBCON` 存在與否用的是最普通的 Open SQL `SELECT`」這種實作細節都親眼確認過，比只看官方文件的說明更扎實。
- 這題順便修正了 `if07` 一個可能造成誤解的地方：`if07` 提到 `DBCON` 用資料預覽工具查會被擋（`You cannot display DBCON with the standard tools`），這題證實那只是資料預覽這個特定便利工具的保護機制，`DBCON` 本身完全是張普通的 Open SQL 可讀表——兩者並不矛盾，只是保護的層級不同（工具層 vs 資料庫權限層），出題或跟學員講解時要把這個細節說清楚，避免學員誤以為 `DBCON` 從程式碼裡也讀不到。
- 情境②刻意用「查詢不存在的表」測試錯誤處理，是最容易穩定重現、不依賴任何外部狀態的失敗案例；更貼近真實情境的失敗（例如 `DBCO` 連線設定錯誤、SQL 語法錯誤、違反主鍵限制）由於這題只用預設連線做唯讀查詢，沒有一併示範，留在思考題讓學員自己設計。

思考題

1. Lecture 提到 ADBC 的 SQL 是字串動態組出來的，那如果一段邏輯用 Open SQL 寫得出來，還需要特地改用 ADBC 嗎？（提示：不需要——ADBC 的價值在於 Open SQL 做不到的情境，例如 `if07` 提到的「連到 `DBCO` 設定的異質資料庫」、資料庫特定函數 / 語法、動態決定要查哪張表這類真正需要跳出 Open SQL 資料庫無關層限制的情境；能用 Open SQL 解決的，優先用 Open SQL，這條原則跟 `if06` 對 Native SQL 的態度一致）
2. `execute_query` 用完 `result_set` 要記得呼叫 `close()`（答案程式裡有做），如果忘記關閉，可能會有什麼後果？（提示：資料庫游標 / 連線資源沒有正常釋放，長時間執行或高頻呼叫的程式可能累積耗用資源，這跟一般程式設計「開了什麼資源就要記得關」的原則一致，只是這裡換成資料庫游標）

3. if06 學到 Native SQL 「新開發已凍結，只能在 ADBC 做」，那如果現在維護一支很舊的、大量用 `EXEC SQL...ENDEXEC` 的程式，要不要藉這次維護的機會整個改寫成 ADBC？（提示：這是典型的技術債取捨問題——如果這次改動範圍本來就要碰到那段程式、且有完整測試覆蓋確保改寫後行為一致，值得順手改；如果只是要修一個小 bug、改動範圍應該盡量小，貿然整段重寫風險反而更高，這跟 CLAUDE.md / team 慣例強調的「不要做超出任務範圍的重構」是同一個原則）